

DLIC HW6

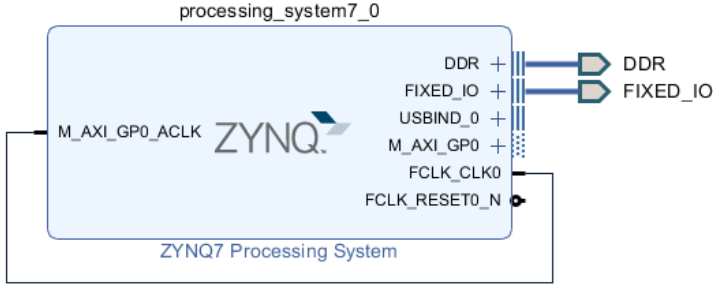
Data Transfer between PC and PYNQ-Z2 through UART Interface

系級:資訊系大四 學號:E34106010 姓名:黃偉峰

1. Homework Result



2. Code Explanation

Vivado Block Diagram
 The diagram shows a ZYNQ7 Processing System block labeled 'processing_system7_0'. It has several input and output ports on its right side: 'DDR' (connected to a DDR memory block), 'FIXED_IO' (connected to a FIXED_IO block), 'USBIND_0', 'M_AXI_GP0', 'FCLK_CLK0', and 'FCLK_RESET0_N'. An input port 'M_AXI_GP0_ACLK' is on the left side. The ZYNQ logo is visible inside the block.
SDK code
Include library
<pre data-bbox="350 909 1352 1178">#include <stdlib.h> #include "xil_types.h" //u8 type #include "xuartps.h" //UART device #include "xparameters.h" //device ID #include "sleep.h"</pre> • <code>stdlib.h</code>: 為了後續使用 <code>malloc</code> 而 include • <code>xil_types.h</code>: 該 header 檔內有 xilinx 定義的各種 data type • <code>xuartps.h</code>: 此為 Xilinx 提供的 UART library，內含 UART 裝置相關的 struct 與 function，可用於初始化 UART、設定 Baud Rate 與進行資料傳輸。 • <code>xparameters.h</code>: 此檔案由 Vivado 自動產生，內含系統中各個硬體模組（如 UART、GPIO、AXI IP）的對應 Device ID、Base Address 等定義。 • <code>sleep.h</code>: 提供 sleep 相關函式，例如 <code>usleep()</code> 可用來讓 CPU 暫停一段時間（單位為微秒）。

Define parameter
<pre>#define BAUDRATE 115200 #define IMAGE_SIZE 512*512 #define HEADERSIZE 1080 #define FILESIZE IMAGE_SIZE + HEADERSIZE</pre> • BAUDRATE：設定 UART 傳輸的 Baud Rate，本程式設為 115200，代表每秒傳輸 115200 bits。 • IMAGE_SIZE：影像大小，此處設為 $512 \times 512 = 262144$ bytes，代表一張 512x512 灰階影像，每個 Pixel 1 byte。 • HEADERSIZE：影像檔案的標頭大小(BMP header)，此處設為 1080 bytes。 • FILESIZE：總接收資料長度 = 標頭資料 + 影像資料，計算出來為 263224 bytes，作為 UART 接收的總目標大小。
ImageData Array
<pre>imageData = malloc(sizeof(u8) * IMAGE_SIZE + HEADERSIZE);</pre> • 使用 imageData malloc 一塊記憶體空間，作為儲存 UART 傳送的資料。配置大小為影像像素資料 ($512 \times 512 = 262144$ bytes) 加上額外的標頭區段 (1080 bytes)，總共 263224 bytes。

UART Initialization

```
XUartPs_Config *myUartConfig;
XUartPs myUart;
myUartConfig = XUartPs_LookupConfig(XPAR_PS7_UART_0_DEVICE_ID);
status = XUartPs_CfgInitialize(&myUart, myUartConfig, myUartConfig->BaseAddress);
if(status != XST_SUCCESS){
    print("UART Initialization FAIL!!");
}

status = XUartPs_SetBaudRate(&myUart, BAUDRATE);
if(status != XST_SUCCESS){
    print("Baud rate Initialization FAIL!!");
}
```

- XUartPs_LookupConfig():
根據 UART 裝置的 ID (通常從 xparameters.h 裡來) 去查詢這個裝置對應的 config struct。
return 一個指向 XUartPs_Config struct 的 ptr, 內部包含:
 - UART 的 Base Address
 - Device ID
- XUartPs_CfgInitialize():
根據 XUartPs_LookupConfig 查到的 Config 和 base address, 去初始化一個 UART 實體。

Set UART Baud Rate

```
status = XUartPs_SetBaudRate(&myUart, BAUDRATE);
if(status != XST_SUCCESS){
    print("Baud rate Initialization FAIL!!");
}
```

- XUartPs_SetBaudRate():
設定 UART 的 Baud Rate, 本程式將 baud rate 設為 115200 bps, 每秒傳輸 115200 bits。若設定失敗, 會回傳錯誤狀態並顯示錯誤訊息。

UART Receiving

```
//Data transfer from Computer to DDR
while(totalRecvBytes < FILESIZE){
    recvBytes = XUartPs_Recv(&myUart, (u8*)&imageData[totalRecvBytes], 100);
    totalRecvBytes += recvBytes;
}
```

- 透過 while loop 調用 XUartPs_Recv()，UART 接收資料後儲存至 imageData array 中。每次最多接收 100 bytes，實際接收量由 recvBytes 回傳，再累加至 totalRecvBytes 記錄已接收總量，下次再從 index 為 totalRecvBytes 的地方開始擺放資料。
- 當接收量達到預定的 FILESIZE (Image Data+ Header) 後，迴圈結束，表示整張影像已成功儲存至記憶體中。

UART Sending

```
//Send Data to the Computer
while(totaltransmittedBytes < FILESIZE){
    transmittedBytes = XUartPs_Send(&myUart, (u8*)&imageData[totaltransmittedBytes], 1);
    totaltransmittedBytes += transmittedBytes;
    usleep(2000);
}
```

- 透過 while loop 調用 XUartPs_Send()逐筆傳輸，每次傳送 1 byte。
- 為避免資料傳送過快造成 buffer 堵塞，每傳送一筆進行 2ms delay，確保資料穩定送達。傳輸量達到 FILESIZE 表示整張影像已傳輸完畢。

Image Inversion

```
//Read Data from DDR, process it, store back in DDR
for(int i = HEADERSIZE; i < FILESIZE; i++){
    imageData[i] = 255 - imageData[i];
}
```

- 從 header 後的 Byte 開始，將每個 Pixel 轉為其反色值，即 $255 - \text{原始值}$ 。

3. Lesson Learn

透過本次實作，我學會了如何使用 Xilinx 提供的硬體驅動程式。LookupConfig 與 CfgInitialize 這兩種 Function 是驅動 ZYNQ 系統內建硬體（如 UART、GPIO、SPI 等）的必要步驟，在 Xilinx SDK 中幾乎所有硬體初始化流程的都是類似的。透過這次 UART 的實作範例，我更加熟悉這一套流程，對於未來操作其他內建周邊裝置時也能更快上手。

另外一開始看到助教提供的 UART 的範例程式碼時，其實完全看不懂程式在做什麼，尤其是初始化流程與各個函式的用途都讓我感到不解，但後來我開始試著去閱讀 Xilinx 官方提供的 header 檔與對應的 .c 檔實作，透過觀察 Function 定義、參數說明與 struct 內容，慢慢釐清了整個驅動流程。